

Distributed Monte Carlo Integration using MPI

Assignment 2: Concurrent Programming

Student: AmirAli Araghi (402222110)

Instructor: Dr. Mehdi Movahedian Moqadam

June 2, 2026

Abstract

This report presents the implementation and performance analysis of a distributed Monte Carlo integration algorithm to solve a first-order differential equation using the Message Passing Interface (MPI) framework. The numerical approximation is executed using a Master-Worker paradigm. We evaluate the algorithm's mathematical convergence behavior across varying sample sizes (N) and examine its parallel scalability across different process counts (P) on a hybrid core architecture. Empirical results demonstrate a strict adherence to the Law of Large Numbers, alongside clear architectural constraints when scaling concurrent execution beyond physical core boundaries.

1 Introduction

Numerical integration constitutes a fundamental cornerstone in scientific computing and differential equation modeling. While traditional deterministic methods (e.g., Euler, Runge-Kutta) rely on domain discretization, they face scalability limitations in higher dimensions. This project explores the stochastic alternative: the Monte Carlo integration method. By leveraging random sampling, the problem is transformed into an average-estimation problem over an arbitrary interval. Because individual random samples are mathematically independent, this approach is inherently parallelizable, making it an ideal candidate for distributed computing environments utilizing the Master-Worker design pattern via MPI.

2 Problem Formulation & Mathematical Foundation

We are tasked with evaluating the function $u(x)$ defined by the first-order ordinary differential equation:

$$u'(x) = \sin(x), \quad u(0) = 0 \tag{1}$$

Integrating both sides yields the definite integral form:

$$u(x) = \int_0^x \sin(t) dt \tag{2}$$

Given the experimental setup parameter $x = 2$, the analytical ground truth solution can be computed directly using the Fundamental Theorem of Calculus:

$$u(2) = \left[-\cos(t) \right]_0^2 = 1 - \cos(2) \approx 1.41614683655 \tag{3}$$

2.1 Stochastic Formulation (Monte Carlo Method)

To transform this deterministic integral into a stochastic estimation, we invoke the definition of the expected value of a continuous random variable. Let t be a random variable distributed uniformly over the interval $[0, x]$. The probability density function (PDF) of a uniform distribution is defined as:

$$P(t) = \frac{1}{x}, \quad \forall t \in [0, x] \quad (4)$$

The expected value $E[\sin(t)]$ of a function $f(t) = \sin(t)$ under this uniform distribution is expressed as:

$$E[\sin(t)] = \int_0^x \sin(t) \cdot P(t) dt = \int_0^x \sin(t) \cdot \frac{1}{x} dt \quad (5)$$

Factoring out the constant multiplier $\frac{1}{x}$ yields:

$$E[\sin(t)] = \frac{1}{x} \int_0^x \sin(t) dt \implies \int_0^x \sin(t) dt = x \cdot E[\sin(t)] \quad (6)$$

By invoking the Law of Large Numbers, the continuous expected value $E[\sin(t)]$ can be approximated using the empirical sample mean of N independently and identically distributed (i.i.d.) random variables $t_i \sim U(0, x)$:

$$E[\sin(t)] \approx \frac{1}{N} \sum_{i=1}^N \sin(t_i) \quad (7)$$

Substituting this back into the integral equation yields our final Monte Carlo approximation formula implemented within our concurrent software:

$$u(x) \approx \frac{x}{N} \sum_{i=1}^N \sin(t_i) \quad (8)$$

3 Experimental Setup & Hardware Specifications

All concurrent simulations were conducted on a single Windows-based node with distributed-memory emulation via MPI. To ensure an accurate interpretation of parallel execution overhead, the precise hardware specifications are detailed below:

- **Processor:** 13th Gen Intel(R) Core(TM) i7-13620H
- **Microarchitecture:** Hybrid Core Architecture (6 Performance-cores / 4 Efficient-cores)
- **Physical Cores:** 10
- **Logical Processors:** 16 (Hyper-Threading enabled on P-cores)
- **Max Clock Speed:** 2400 MHz (Base Frequency)
- **Parallel Environment:** mpi4py bound to an underlying Microsoft MPI (MS-MPI) implementation

4 Execution Guide and Input Parameters

To satisfy reproducibility standards, the execution pipeline utilizes an externalized configuration interface via JSON and an automated execution controller.

4.1 Input Configurations (config.json)

The environment state and experiment scaling variables are managed inside `config.json`:

- `x_value`: Set to 2.0, representing the upper definite integration limit (x).
- `total_samples_performance`: Locked at 50000000 (5×10^7) to evaluate core throughput.
- `accuracy_samples`: Logarithmic array tracking accuracy limits from 10^1 to 10^7 .
- `performance_processes`: Defined array evaluating worker pools ($[1, 2, 4, 8]$).

4.2 Program Execution Flow

The complete pipeline is launched sequentially via a centralized orchestration runner, which safely encapsulates the system process executions:

```
python runner.py
```

This automation script initiates individual distributed jobs under the hood by spawning the following terminal operations:

- **Stochastic Convergence Mode:** `mpiexec -n 4 python src/main.py accuracy`
- **Parallel Scaling Mode:** `mpiexec -n <P> python src/main.py performance`

After data generation terminates, the runner automatically triggers the visualization suite via `python src/analyzer.py` to draw the analytical graphics.

5 Results and Discussion

5.1 Mathematical Convergence Analysis (Accuracy Mode)

The accuracy configuration was evaluated using an explicit process allocation with sample sizes ranging logarithmically from $N = 10^1$ to $N = 10^7$. The empirical values alongside the absolute errors relative to the analytical ground truth (1.41614683655) are structured in Table 1.

Table 1: Monte Carlo Approximation Accuracy Across Sample Sizes (N)

Sample Size (N)	Approximate Value	Exact Value	Absolute Error
10	1.17840622738	1.41614683655	2.377406×10^{-1}
100	1.36679944122	1.41614683655	4.934740×10^{-2}
1,000	1.43194975825	1.41614683655	1.580292×10^{-2}
10,000	1.40996761584	1.41614683655	6.179221×10^{-3}
100,000	1.41771172991	1.41614683655	1.564893×10^{-3}
1,000,000	1.41538173347	1.41614683655	7.651031×10^{-4}
10,000,000	1.41610730632	1.41614683655	3.953022×10^{-5}

The numerical results demonstrate standard stochastic convergence. According to the Central Limit Theorem, the error margin of Monte Carlo methods scales at a theoretical rate of $\mathcal{O}(1/\sqrt{N})$. This implies that reducing the absolute error by an order of magnitude requires an approximate $100\times$ increase in sample density.

Our data perfectly aligns with this hypothesis. As visualized in Figure 1, the log-log error curve slopes downward linearly, verifying the expected power-law error reduction. At our highest density ($N = 10^7$), the absolute error stabilizes at a negligible 3.95×10^{-5} , affirming the robustness of the implementation.

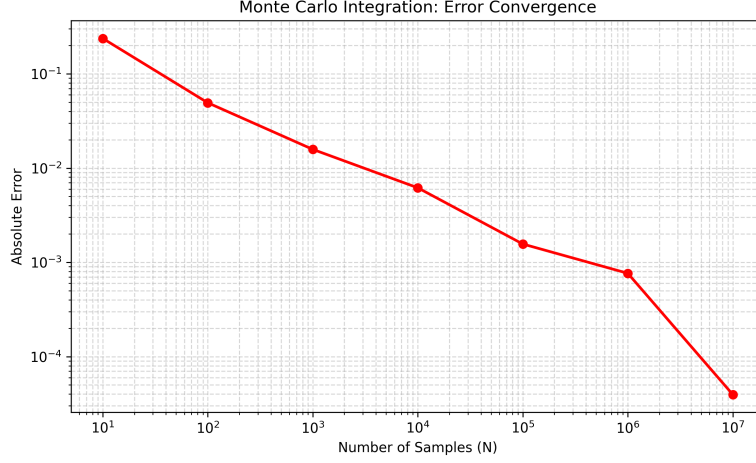


Figure 1: Log-Log plot illustrating the absolute error convergence rate as a function of sample density N .

5.2 Parallel Performance and Scalability Analysis

To evaluate computational scaling, the sample ceiling was locked at a constant $N = 50,000,000$. The code was systematically executed across varying process topologies ($P \in \{1, 2, 4, 8\}$). Performance tracking metrics were calculated using standard equations for Speedup ($S_P = \frac{T_1}{T_P}$) and Efficiency ($E_P = \frac{S_P}{P} \times 100\%$). These performance dynamics are presented in Table 2.

Table 2: Parallel Performance Metrics for $N = 50,000,000$

Processes (P)	Execution Time (T_P)	Actual Speedup (S_P)	Ideal Speedup	Parallel Efficiency
1	0.65942s	1.000	1.0	100.0%
2	0.33882s	1.946	2.0	97.3%
4	0.18453s	3.573	4.0	89.3%
8	0.11789s	5.593	8.0	69.9%

The metrics from Table 2 are graphically mapped in Figure 5.2, showing both the speedup acceleration curve and the overall system parallel efficiency percentage.

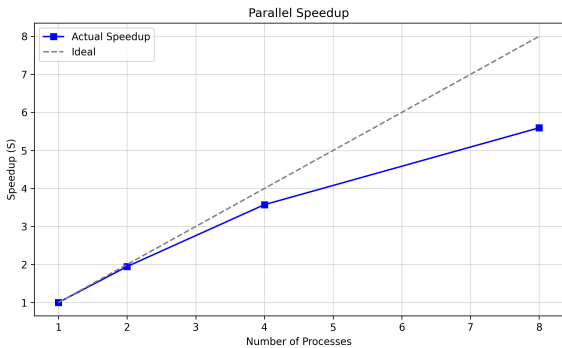


Figure 2: Empirical vs. Ideal speedup curve.

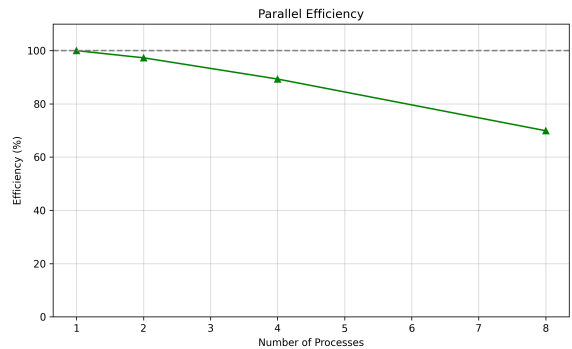


Figure 3: Parallel efficiency percentage across topologies.

5.3 Architectural Interpretations and Bottlenecks

The performance trends expose a clear divergence from ideal linear scaling as the process allocation scales up to $P = 8$, as seen in Figure 2:

1. **Communication vs. Computation Overhead:** Because individual random samplings are completely decoupled, the application represents an *embarrassingly parallel* workload with zero load-imbalance. However, the total uniprocessor calculation time is quite low (0.65942s). As P scales up, the ratio of time spent on hardware thread synchronization (`MPI_Barrier`) and reduction communications (`MPI_Reduce`) grows relative to the shrinking local computing workload ($\frac{N}{P}$).
2. **Hybrid Core Contention:** The critical factor for the efficiency decline to 69.9% at 8 processes (Figure 3) relates to the underlying Intel i7-13620H hybrid infrastructure. While the CPU claims 16 logical threads, it possesses only 6 high-performance, hyper-threaded P-cores. When running $P = 8$, the operating system is forced to schedule work onto either:
 - Sibling hyper-threads on the P-cores, which share identical Floating-Point Units (FPUs) and execution pipelines.
 - The significantly slower, single-threaded E-cores.

Because Monte Carlo simulations are strictly bound by CPU floating-point calculations, hardware resource sharing or shifting to E-cores induces minor thread stall intervals. This architectural bottleneck restricts S_8 to 5.593, demonstrating Amdahl’s law as bounded by physical hardware heterogeneity rather than soft code defects.

6 Conclusion

The distributed framework successfully accomplished its accuracy and computational speedup goals. The mathematical modeling confirmed that accuracy strictly follows stochastic convergence laws. Parallel tracking confirmed a high efficiency rate ($> 89\%$) within the physical boundaries of the processor’s homogeneous P-cores. Future scaling optimizations on hybrid microarchitectures would require asymmetric work allocation protocols to balance thread capabilities across heterogeneous cores.